

Part01:

GitHub:

<https://github.com/janaashraf888/rowadmasrtasks.git>

1. Why is it better to code against an interface rather than a concrete class?

Coding against an interface rather than a concrete class improves flexibility, maintainability, and scalability. It allows programs to depend on behavior instead of specific implementations, making it easier to replace or modify classes without affecting the rest of the system. This approach supports loose coupling, enhances testability, and follows good object-oriented design principles such as dependency inversion.

2. When should you prefer an abstract class over an interface?

An abstract class should be preferred when related classes share common behavior, fields, or base functionality. It is useful when partial implementation is needed, such as providing default methods or protected members. Abstract classes are suitable when strong inheritance relationships exist and when future extensions may require shared code.

3. How does implementing Comparable improve flexibility in sorting?

Implementing Comparable allows objects to define their own natural ordering. This enables collections to be sorted automatically using built-in sorting methods without requiring

external comparison logic. It improves flexibility by allowing custom objects to be sorted consistently and reused easily in different parts of the application.

4. What is the primary purpose of a copy constructor in C#?

The primary purpose of a copy constructor is to create a new object as a duplicate of an existing one. It ensures that the new object has the same values as the original while maintaining independent memory references. This is especially important for avoiding unintended side effects when working with reference-type fields.

5. How does explicit interface implementation help in resolving naming conflicts?

Explicit interface implementation helps resolve naming conflicts when multiple interfaces contain methods with the same signature. By implementing interface methods explicitly, a class can provide separate implementations for each interface. These methods can only be accessed through the interface reference, ensuring clarity and preventing ambiguity.

6. What is the key difference between encapsulation in structs and classes?

In classes, encapsulation is based on reference types, meaning objects are accessed through references and stored on the heap. In structs, encapsulation is based on value types, meaning data is stored directly and copied when passed. This affects memory

management, performance, and behavior when modifying objects.

7. What is abstraction as a guideline, and what is its relation with encapsulation?

Abstraction is a guideline that focuses on exposing only essential features while hiding unnecessary details. It defines what an object can do without revealing how it does it. Encapsulation supports abstraction by protecting internal data and implementation through access modifiers. Together, they help create clear, secure, and maintainable designs.

8. How do default interface implementations affect backward compatibility in C#?

Default interface implementations allow interfaces to provide method bodies. This enables developers to add new methods to existing interfaces without breaking classes that already implement them. As a result, backward compatibility is maintained, and existing code continues to work without modification.

9. How does constructor overloading improve class usability?

Constructor overloading improves class usability by allowing objects to be created in multiple ways using different parameter sets. It provides flexibility for developers to initialize objects

according to their needs. This reduces complexity, improves readability, and makes class usage more intuitive.

Part06:

2. What do we mean by coding against an interface rather than a class?

What do we mean by coding against abstraction, not concreteness?

Coding against an interface rather than a concrete class means that a program depends on an abstract definition (interface) instead of a specific implementation. The code interacts with what an object can do, not how it is implemented.

When we say “code against abstraction, not concreteness,” we mean that classes should depend on general contracts (interfaces or abstract classes) rather than fixed classes. This makes the system more flexible, because implementations can be changed or replaced without modifying the main code.

This approach reduces tight coupling, improves maintainability, increases reusability, and makes testing easier. It also follows good design principles such as loose coupling and dependency inversion.

3. What is abstraction as a guideline, and how can we implement it through what we have studied?

Abstraction as a guideline means focusing on what a system should do instead of how it does it. It encourages developers to

expose only essential features and hide internal details. The goal is to simplify usage and reduce complexity.

We can implement abstraction through:

- **Interfaces:** By defining method signatures without implementation, allowing different classes to provide their own behavior.
- **Abstract Classes:** By providing partial implementations and forcing derived classes to complete the remaining behavior.
- **Access Modifiers:** By using private, protected, and public members to hide internal logic.
- **Encapsulation:** By grouping data and related methods together and controlling access to them.

Through these techniques, abstraction helps create clean, understandable, and maintainable software designs where users of a class only care about what it does, not how it works internally.

4. What is Operator Overloading?

Operator overloading is a feature in C# that allows programmers to redefine the behavior of operators (such as +, -, *, ==, etc.) for user-defined types like classes and structs.

By using operator overloading, objects can be manipulated using standard operators in a natural and readable way. For example, two objects can be added or compared using operators instead of calling special methods.

This improves code readability, makes programs easier to understand, and allows custom types to behave like built-in data

types. However, operator overloading should be used carefully to avoid confusion and maintain clear program logic.

LinkedIn Post:

https://www.linkedin.com/posts/janaashraf88_csharp-dotnet-programming-activity-7429266460679643136-aKKT?utm_source=share&utm_medium=member_desktop&rcm=ACoAAE8YvOUBdlbw_Me3Yv_nmlqA8meTZp5UEhc